

Spot the Fake Photo — Submission Note

Approach

Rather than a heavy deep learning model, I built a 26-feature extraction pipeline (OpenCV + NumPy) paired with a Gradient Boosting Classifier (300 trees, scikit-learn). Features target the physical artifacts of screen recapture: Moire patterns (FFT-based), sensor noise residuals, blue-channel bias from backlit screens, texture uniformity, and unique colour ratio. The trained model is saved via joblib and loaded directly at inference time — no linear approximation.

To work around the small dataset (50 real + 50 screen), each image was augmented with 5 variants (brightness, contrast, blue tint, zoom), expanding training to 600 samples. StratifiedGroupKFold was used in cross-validation to prevent leakage between augmented copies of the same image.

Accuracy

- 5-Fold CV: 88.3% (+/- 3.7%) — measured on real phone-recaptured photos of screens at varying distances, angles, and lighting; not trivial digital screenshots
- Held-out: 95.0% — 6 errors out of 120 test images (5 real flagged as recapture, 1 recapture missed)

Latency & Cost

- Latency: ~65ms per image on a standard CPU (images downsampled to 512px before processing)
- Server cost: ~\$1.00 per 1M images on AWS Lambda (1024MB, 65ms/image at \$0.0000001 per 100ms)

What I'd improve with more time

1. Tune the decision threshold using a Precision-Recall curve to guarantee <0.1% False Positive Rate — flagging honest users is worse than missing a cheater
2. Collect 500+ recaptures across device types and print media to tighten CV variance
3. Add text-density detection to catch high-res monitor recaptures where Moire is too subtle